

Contents

	1
(CDI System)	1
.	1
.	1
Backend -	3
1. server.py ()	3
2. security_utils.py ()	4
3. security_routes.py ()	4
Frontend -	4
1. (Pages)	4
2. (Components)	5
3. (Contexts)	6
-	6
1. JWT Implementation	6
2. Password Hashing (bcrypt)	6
3. Rate Limiting	6
4. Input Validation (Pydantic)	6
-	7
Google Gemini Integration	7
-	7
MongoDB Collections	7
.	8
Backend (requirements.txt)	8
Frontend (package.json)	8
.	8
.	8
.	9

(CDI System)

: 2025

** : ** 100+
** : ** 15,000+
: Python, JavaScript/React, HTML, CSS
** : ** MongoDB

/app/	
backend/	# Backend (Python/FastAPI)
server.py	# 3,700+ lines
security_models.py	# 200+ lines
security_utils.py	# 500+ lines
security_routes.py	# 600+ lines
clinical_questions.py	# 150+ lines

```

migration_update_users.py      # 300+ lines
send_password_change_emails.py # 100+ lines
requirements.txt               # Dependencies
.env                           # Environment variables

frontend/                      # Frontend (React.js)
  public/
    index.html
    logo.jpeg
    download-2.png
  src/
    components/
      ui/                      # 20+ Shadcn components
        button.jsx
        card.jsx
        input.jsx
        dialog.jsx
        select.jsx
        table.jsx
        badge.jsx
        ...
        Navbar.jsx            # 200+ lines
        Footer.jsx            # 100+ lines
        ChatWidget.jsx        # 400+ lines
        WhatsAppSupport.jsx   # 150+ lines
    contexts/
      LanguageContext.jsx     # 100+ lines
    hooks/
      use-toast.js            # 50+ lines
    utils/
      errorHandler.js         # 115+ lines
    pages/
      Login.jsx               # 300+ lines
      Register.jsx            # 400+ lines
      MFAVerification.jsx     # 250+ lines
      Dashboard.jsx           # 500+ lines
      NewNote.jsx             # 600+ lines
      Analysis.jsx            # 700+ lines
      History.jsx             # 400+ lines
      Chat.jsx                # 500+ lines
      ChatEnhanced.jsx        # 800+ lines
      AdminDashboard.jsx       # 600+ lines
      AdminDashboard_v2.jsx    # 800+ lines
      SupervisorDashboard.jsx  # 500+ lines
      SupervisorDashboardPro.jsx # 700+ lines
      Messages.jsx            # 400+ lines
      EditNote.jsx            # 500+ lines
      ForgotPassword.jsx       # 300+ lines
      ResetPassword.jsx        # 300+ lines
      SecurityDashboard.jsx    # 400+ lines
    App.js                    # 400+ lines
    App.css                   # 200+ lines
    index.js                  # 50+ lines
    index.css                 # 100+ lines
    i18n.js                   # 150+ lines
  package.json                # Dependencies

```

```

tailwind.config.js      # Tailwind config
postcss.config.js       # PostCSS config
.env                    # Environment variables

documentation/          #
  EXECUTIVE_SUMMARY_AR.md      #
  TECHNICAL_DOCUMENTATION_AR.md #
  INTELLECTUAL_PROPERTY_PATENT_AR.md#
  CYBERSECURITY_DOCUMENTATION_AR.md #
  CODE_INVENTORY_AR.md        #
  ADMIN_CREDENTIALS.txt
  SECURITY_DOCUMENTATION.md
  DEPLOYMENT_CHECKLIST_AR.md
  FINAL_ADMIN_ACCOUNT_INFO.md
  ... (20+ )

scripts/                #
  update_admin_email.sh
  post_deployment.sh
  ...

```

Backend -

1. server.py ()

: 3,700+

** .**

```

# API Endpoints
@app.post("/api/auth/login-step1")      # - 1
@app.post("/api/auth/login-step2")      # MFA - 2
@app.post("/api/auth/register")          #
@app.post("/api/notes/create")           #
@app.post("/api/analysis/analyze")       # AI
@app.post("/api/chat/message")           # AI
@app.get("/api/admin/statistics")         #
@app.get("/api/admin/audit-logs")        #

# Security Functions
async def ensure_admin_account()         #
async def verify_token()                 # JWT
async def check_rate_limit()             # Rate limiting
async def audit_log()                    #

# AI Integration
async def analyze_clinical_note()        #
async def chat_with_ai()                 #
async def generate_physician_query()     #

# Database Operations
async def get_user()                     #
async def create_note()                  #
async def save_analysis()                #

```

: - FastAPI framework - Async/await patterns - Pydantic validation - MongoDB motor driver - Google Gemini AI
 - JWT authentication - Bcrypt password hashing

2. security_utils.py ()

: 500+
:

```
# Authentication
def create_access_token()           # JWT token
def verify_token()                  #
def hash_password()                 #
def verify_password()               #

# MFA
def generate_otp()                  # OTP
async def send_otp_email()          # OTP
async def verify_otp()              # OTP

# Rate Limiting
async def check_rate_limit()        #
async def increment_attempts()      #
async def unlock_account_if_expired() #

# Email
async def send_email()              #
async def send_welcome_email()      #
async def send_password_reset_email() #
```

3. security_routes.py ()

: 600+
:

```
@router.post("/security/mfa/send-otp")  # OTP
@router.post("/security/mfa/verify-otp") # OTP
@router.post("/security/password/change") #
@router.post("/security/password/reset") #
```

Frontend -

1. (Pages)

Login.jsx () : 300+

```
// Features:
- Multi-language support (AR/EN)
- Email + Password validation
- MFA flow integration
- Error handling with toast
- Remember me functionality
- Loading states
```

Dashboard.jsx () : 500+

```
// Features:
- User statistics display
- Recent notes listing
- Quick actions panel
- Activity charts
- Responsive design
```

NewNote.jsx () : 600+

// Features:

- Rich text editor
- Patient info form
- Privacy warning dialog
- Real-time validation
- Auto-save draft
- Submit to AI analysis

Analysis.jsx () : 700+

// Features:

- Display AI analysis results
- ICD-10-CM codes with descriptions
- Gaps identified
- Physician queries
- CDI notes
- Export to PDF
- Edit & re-analyze option

ChatEnhanced.jsx () : 800+

// Features:

- Two chat modes (clinical questions / open chat)
- Specialty selection
- Context-aware responses
- Message history
- Real-time typing indicator
- Copy responses
- Print conversation

AdminDashboard.jsx () : 600+

// Features:

- User management (CRUD)
- System statistics
- Audit logs viewer
- Role assignment
- Account activation/deactivation
- Bulk operations

2. (Components)

Navbar.jsx () : 200+

// Features:

- Responsive menu
- Language switcher
- User dropdown
- Notifications
- Logout

ChatWidget.jsx () : 400+

// Features:

- Floating chat button
- Expandable chat window
- AI-powered responses

- Message history
- Minimize/maximize

3. (Contexts)

LanguageContext.jsx () : 100+

```
// Features:
- i18n integration
- Language switching
- RTL support
- Translation management
```

-

1. JWT Implementation

```
# Token generation with 256-bit secret
SECRET_KEY = os.environ['SECRET_KEY'] # 256-bit random key
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_HOURS = 24

def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(hours=24)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

2. Password Hashing (bcrypt)

```
import bcrypt

def hash_password(password: str) -> str:
    salt = bcrypt.gensalt(rounds=12) # 12 iterations
    hashed = bcrypt.hashpw(password.encode('utf-8'), salt)
    return hashed.decode('utf-8')
```

3. Rate Limiting

```
@app.post("/api/auth/login-step1")
@limiter.limit("5/minute") # 5 attempts per minute
async def login(credentials: UserLogin):
    # Implementation
    pass
```

4. Input Validation (Pydantic)

```
class UserRegister(BaseModel):
    email: EmailStr # Auto email validation
    password: str
    full_name: str

    @validator('password')
    def validate_password(cls, v):
        if len(v) < 8:
            raise ValueError('Password too short')
        # ... more checks
        return v
```

-

Google Gemini Integration

```
import google.generativeai as genai

genai.configure(api_key=GEMINI_API_KEY)
model = genai.GenerativeModel('gemini-2.0-flash-exp')

async def analyze_note(note_text: str):
    prompt = f"""
        :
        {note_text}

        :
        1.
        2. ICD-10-CM
        3.

        JSON format
        """

    response = model.generate_content(prompt)
    return json.loads(response.text)
```

-

MongoDB Collections

```
// Users
{
  id: UUID,
  email: String,
  password_hash: String,
  role: String,
  mfa_enabled: Boolean,
  // ... 15+ fields
}

// Clinical Notes
{
  id: UUID,
  user_id: String,
  patient_id: String,
  note_text: String,
  // ... 10+ fields
}

// Analyses
{
  id: UUID,
  note_id: String,
  gaps: Array,
  icd10_codes: Array,
```

```

    // ... 8+ fields
}

// Audit Logs
{
    user_id: String,
    action: String,
    timestamp: Date,
    // ... 8+ fields
}

```

Backend (requirements.txt)

```

fastapi==0.104.1
uvicorn==0.24.0
motor==3.3.2
pymongo==4.6.0
pydantic==2.5.0
python-jose==3.3.0
bcrypt==4.1.1
google-generativeai==0.3.1
aiosmtplib==3.0.1
python-dotenv==1.0.0
slowapi==0.1.9
prometheus-client==0.19.0

```

Frontend (package.json)

```

{
  "react": "^18.2.0",
  "react-router-dom": "^6.20.0",
  "axios": "^1.6.2",
  "i18next": "^23.7.6",
  "react-i18next": "^13.5.0",
  "@radix-ui/react-*": "^1.0.0",
  "tailwindcss": "^3.3.0",
  "lucide-react": "^0.294.0",
  "sonner": "^1.2.0"
}

```

(EXECUTIVE_SUMMARY_AR.md) (TECHNICAL_DOCUMENTATION_AR.md) (INTELLECTUAL_PROPERTY_PATENT_DOCUMENTATION_AR.md) (CYBERSECURITY_DOCUMENTATION_AR.md) (CODE_INVENTORY_AR.md)

Backend	10+	6,000+	Python
Frontend	50+	8,000+	JavaScript/React
UI Components	30+	1,000+	JSX/CSS
Documentation	25+	-	Markdown

****	115+	15,000+	****
------	------	---------	------

: /app/ ()

** .*

#

tar -czf cdi-system-complete.tar.gz /app/

Git

git clone /app/.git cdi-system-backup

: .

** 2025 CDI System - **